

News Category Word Cloud Analysis

Methodology

For each article category (Business, Entertainment, Health, Technology), a WordCloud was generated using all articles belonging to that category.

The preprocessing steps were:

- Concatenation of Title + Content into a single text per article.
- Lowercasing of all text.
- Removal of URLs and email addresses.
- Removal of numbers and punctuation (keeping only a–z characters).
- Removal of English stopwords and additional news-specific stopwords.
- Removal of words with length < 3.
- Additional removal of very frequent, non-informative global words (AUTO_STOP).
- WordCloud generated using word frequency (size $\propto$ frequency).

The WordCloud size of each word is proportional to its frequency within the specific category corpus.

Business



The dominant terms include stock, market, sales, growth, profit, quarter, business, company, indicating a strong focus on financial markets, corporate performance, and economic activity. The category clearly reflects market movements, earnings reports, and business growth.

Entertainment



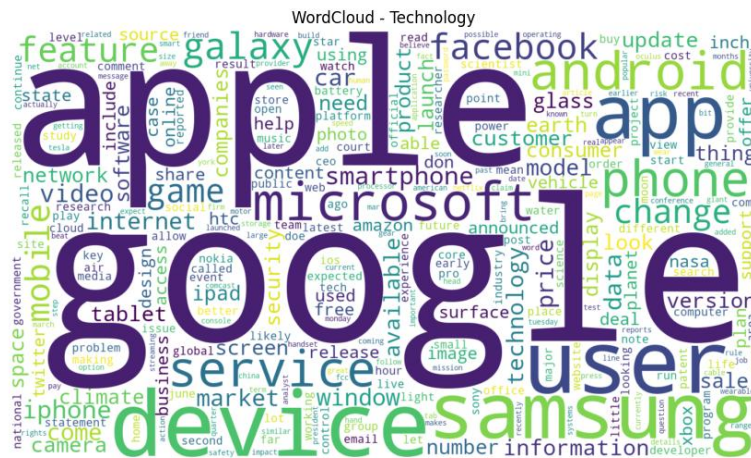
Frequent terms such as film, movie, star, season, music, actor, award, series highlight content related to cinema, television, celebrities, and the entertainment industry. The vocabulary reflects media coverage and popular culture topics.

Health



Key words include cases, virus, disease, drug, medical, patient, hospital, research, indicating coverage of public health issues, medical research, infectious diseases, and clinical topics. The category clearly centers around healthcare and epidemiology.

Technology



Prominent terms such as google, microsoft, app, phone, service, device, software, technology show a focus on major tech companies, digital platforms, consumer electronics, and software innovation. The vocabulary reflects developments in the technology sector.

News Category Classification

Evaluation Results

Statistic Measure	SVM (BoW)	KNN with Jaccard	Optimized Model
Accuracy (5-fold CV)	0.96477	0.96821	0.97537

We evaluated two baseline classifiers using Bag of Words (BoW) features and 5-fold Stratified Cross-Validation with Accuracy. The BoW + Linear SVM baseline achieved 0.96477 mean accuracy, while KNN with Jaccard similarity (binary BoW) achieved 0.96821. To beat the benchmark, we experimented with alternative feature weighting and preprocessing and used **TF-IDF** (unigrams + bigrams, sublinear TF) with a tuned **Linear SVM**, which improved performance to **0.97537** mean accuracy. This indicates that **TF-IDF weighting and sublinear term** frequency reduce the impact of very frequent terms and improve separability for linear margins.

Beat the Benchmark

- **Feature change:** BoW $\rightarrow$ **TF-IDF** to down-weight common terms and emphasize informative terms.
- **Sublinear TF:** `sublinear_tf=True` applies $1 + \log(\text{tf})$, limiting dominance of repeated words.
- **Context features:** `ngram_range=(1,2)` captures informative bigrams (e.g., “stock market”, “health care”).
- **Model tuning:** LinearSVC with `C=1.5` and `max_iter=5000` for stable optimization and improved accuracy.

Finally, we trained **the best-performing model (TF-IDF + Linear SVM)** on the full train set and generated the prediction output file `testSet_categories.csv` with columns `Id`, `Predicted` (comma-separated) for all test instances, which was generated predictions for the unlabeled test set.

Scalable Nearest-Neighbor Search with MinHash LSH

Brute-Force Baseline

The brute-force approach computes the exact Jaccard similarity between every test document and every training document.

Time complexity:

$$O(|Test| \cdot |Train|)$$

Properties:

- Exact similarity computation
- Recall = 1.0 (by definition)
- Very high query time

This serves as the gold standard reference. Also, it is good to clarify that metric *Recall* is computed as:

$$\text{Recall} = \frac{\text{LSH top-K} \cap \text{brute top-K}}{K}$$

LSH with Explicit Threshold = 0.9 (As Required)

In this first implementation:

MinHashLSH(num_perm=num_perm, threshold=0.9)
--

The library automatically selects (b,r) such that the S-curve knee is near 0.9.

Thus:

$$P(s) = 1 - (1 - s^r)^b$$

becomes large only when:

$$s \approx 0.9$$

Practical Interpretation

1. Only document pairs with high expected similarity are returned as candidates.
2. Exact Jaccard similarity is computed only for those candidates.
3. Query time is significantly reduced.
4. Recall drops substantially because true neighbors with $s < 0.9$ are filtered out.

Fallback Case (num_perm = 16)

For num_perm = 16 and threshold = 0.9, there is no integer pair (b, r) satisfying:

$$b \cdot r = 16 \quad \text{and} \quad b \geq 2$$

that yields an effective threshold close to $\tau = 0.9$.

Consequently, the library activates a fallback mechanism: it selects the closest feasible (b, r) configuration that satisfies the structural constraints, even though the resulting S-curve cannot enforce the requested threshold strictly.

This justifies the label “**fallback_params**” in the *Parameters* column of the corresponding table row (see Table 1).

LSH without Explicit Threshold (Fixed $r = 2$)

In this second implementation:

$$r = 2, \quad b = \frac{\text{num_perm}}{2}$$

The probability that a pair with true Jaccard similarity s becomes a candidate is:

$$P(s) = 1 - (1 - s^r)^b$$

The effective threshold (knee of the S-curve) is approximated by:

$$s_0 \approx \left(\frac{1}{b}\right)^{1/r}$$

Effective Threshold Values ($r = 2$)

num_perm	r	b	Effective threshold s_0
16	2	8	$1/\sqrt{8} \approx 0.354$
32	2	16	$1/\sqrt{16} = 0.25$
64	2	32	$1/\sqrt{32} \approx 0.177$

Why Does $r = 2$ Yield the Highest Recall?

From:

$$P(s) = 1 - (1 - s^r)^b$$

When r is small:

- s^r does not shrink dramatically
- Therefore, $P(s)$ remains relatively large even for moderate s
- More pairs become candidates

Consequences:

- Higher recall
- More candidate comparisons
- Larger query time

If r increases:

- s^r decreases sharply (for $s < 1$)
- $P(s)$ decreases
- Fewer candidates are generated
- Recall drops

Therefore, with $r = 2$ (the smallest valid value with $b \geq 2$), the LSH is the loosest configuration and produces the highest recall.

Comparative Summary

- ✓ In all experiments, $K = 7$ nearest neighbors are used.
- ✓ The similarity metric used throughout the experiments is the **Jaccard** similarity.
- ✓ **Both** LSH configurations use identical preprocessing, vectorization, and evaluation procedures to ensure a fair comparison.

Method	Threshold	Recall	Time
Brute Force	–	1.0	Very high
LSH ($r = 2$)	~0.17–0.35	Higher	Moderate
LSH (threshold = 0.9)	0.9	Very low	Very low

Theoretical Insight

The behavior of LSH is governed by:

$$P(s) = 1 - (1 - s^r)^b$$

- r controls sharpness (strictness)
- b controls the slope
- The effective threshold is approximated by:

$$s_0 \approx \left(\frac{1}{b}\right)^{1/r}$$

Small $r \rightarrow$ loose LSH $\rightarrow$ high recall

Large $r \rightarrow$ strict LSH $\rightarrow$ low recall

Imposing threshold = 0.9 enforces a strict S-curve, significantly reducing candidate generation.

Table 1. Performance comparison between brute-force Jaccard similarity and MinHash LSH-based K-NN classification for different numbers of permutations and threshold configurations. Reported metrics include index construction time in seconds (BuildTime), query time in seconds (QueryTime), total runtime (in seconds), recall (fraction of true K nearest neighbors retrieved), and the corresponding LSH parameters (num_perm, threshold, b, r).

Type	BuildTime	QueryTime	TotalTime	Recall	Parameters
Brute-Force-Jaccard	0.000000	182.606849	182.606849	1.000000	–
LSH-Jaccard (Perm=16)	35.966267	16.820395	52.786663	0.025207	num_perm=16, $\tau=0.9$, mode=fallback_params
LSH-Jaccard (Perm=32)	40.440236	18.087601	58.527837	0.015827	num_perm=32, $\tau=0.9$, (b,r)=(2,16)
LSH-Jaccard (Perm=64)	54.015925	23.557514	77.573439	0.014861	num_perm=64, $\tau=0.9$, (b,r)=(3,21)
LSH-Jaccard (Perm=16)	36.646482	43.539663	80.186685	0.300968	num_perm=16, r=2, b=8
LSH-Jaccard (Perm=32)	45.021401	68.034477	113.055878	0.445161	num_perm=32, r=2, b=16
LSH-Jaccard (Perm=64)	63.312638	134.708555	198.021194	0.639589	num_perm=64, r=2, b=32

Custom Dynamic Time Warping Implementation

Mathematical Formulation

Let:

- $A = (\alpha_1, \alpha_2, \dots, \alpha_n)$
- $B = (b_1, b_2, \dots, b_n)$

The local cost between two points is defined as:

$$c(i, j) = |a_i - b_j|$$

The DTW cumulative cost matrix is defined recursively as:

$$dp(i, j) = c(i, j) + \min \begin{cases} dp(i-1, j) \\ dp(i, j-1) \\ dp(i-1, j-1) \end{cases}$$

with boundary conditions:

$$dp(0, 0) = 0$$

$$dp(i, 0) = \infty \quad \text{for } i > 0$$

$$dp(0, j) = \infty \quad \text{for } j > 0$$

The DTW distance is:

$$DTW(A, B) = dp(n, m)$$

Implementation Details

1. Parsing

The dataset contains time series stored as string representations of Python lists. Each string is safely converted to a list of floats using *ast.literal_eval*.

2. Dynamic Programming Optimization

Instead of storing the full $(n + 1) \times (m + 1)$ matrix, only two rows are stored:

- *prev* → previous row
- *curr* → current row

This reduces memory complexity from:

$$O(nm)$$

to:

$$O(m)$$

Time complexity remains:

$$O(nm)$$

3. Edge Cases

- If both sequences are empty → distance = 0
- If one sequence is empty → distance = infinity

4. Execution Time

The total execution time required to compute DTW distances for all rows of the dataset was measured using:

<code>time.perf_counter()</code>

Results:

- Total rows processed: **1002**
- Total computation time: **129.139586 seconds**

This corresponds to full evaluation of the test dataset.

5. Output File

The output file *dtw.csv* was generated in CSV format with the following structure:

id, DTW distance

6. Conclusion

A fully custom implementation of Dynamic Time Warping was developed without the use of external DTW libraries.

The implementation:

- Computes DTW distances
- Uses Euclidean local cost
- Handles sequences of different lengths
- Measures total execution time
- Produces correctly formatted CSV output
- Optimizes memory usage to $O(m)$

Code Availability

The full implementation is provided as separate Python scripts in the repository. The scripts include text preprocessing, word cloud generation, news classification, MinHash LSH similarity search, and a custom Dynamic Time Warping implementation.